

National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Faculty of Computing

CS110: Fundamentals of Computer Programming

Lab 07: Functions in C++

Skill Builder

Class: BSAI / BSDS - Fall 2026

CLO 1: Apply appropriate programming concepts and techniques to solve simple real-world problems

CLO 3: Build a program and associated documentation using appropriate IDE and supplementary tools

Time: 3 Hours

Instructor: Ms. Sahar Arshad
Lab Engineer: _____

Learning Objectives

After completing this lab, you will be able to:

- Recognize and correctly use a function prototype/declaration, function definition, and function call.
- Differentiate parameters from arguments and choose suitable return types such as void, int, double, and bool.
- Use built-in library functions inside programmer-made functions where appropriate.
- Construct programs using functions with no arguments/no return, arguments/no return, and arguments/return values.
- Trace call-by-value behavior and apply a simple reference parameter when a function must modify the original variable.
- Break a larger problem into small reusable functions.

Function Layout to Use in This Lab

Unless a task says otherwise, use the beginner layout practiced in the lecture:

```
// 1. Include headers
// 2. Function prototypes
int main() {  // 3. Function calls inside main()  }
// 4. Function definitions below main()
```

Important: Do not use global variables. Give every function one clear job. All four tasks are compulsory.

Lab Tasks

Task 1 [CLO 1] - CS110 Console Banner and Reuse [BASIC | 25 minutes]

Start with the simplest function category: no arguments and no return value. The purpose is to master prototype -> call -> definition and observe how control returns to main().

Required function:

```
void printDivider();
```

1. Write the prototype before main(). Define printDivider() below main().
2. Inside printDivider(), use a loop to print exactly 30 hyphens (-) on one line and then move to the next line.
3. In main(), call printDivider() three times to format this console output: a divider, the title "CS110 FUNCTION PRACTICE", another divider, the line "Prototype -> Call -> Definition", and a final divider.
4. Do not copy the 30-hyphen printing code into main(); the repeated work must be performed by the function.
5. In your report, identify one prototype, one function call, and the function definition. In one sentence explain where execution continues after printDivider() finishes.

Skill check: If the definition remains below main() but the prototype is removed, explain why the call may fail to compile.

Deliverable: Complete C++ program, one output screenshot, and the short anatomy/control-flow explanation.

Task 2 [CLO 1] - Campus Shuttle Distance Helper [EASY | 35 minutes]

A campus navigation program needs the straight-line distance between two map points and a short travel suggestion. This task combines a built-in library function with two programmer-made functions.

Required functions:

```
double calculateDistance(double x1, double y1, double x2, double y2);
void printTravelAdvice(double distance);
```

1. Read x1, y1, x2, and y2 as double values in main().
2. In calculateDistance(), calculate and return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2)). Include <cmath>.
3. Store the returned distance in main() and display it clearly.
4. Pass the distance to printTravelAdvice(). Print "Walk" when distance <= 1.0, "Use campus shuttle" when distance <= 5.0, otherwise print "Use transport".
5. Required Test A: (0, 0) to (3, 4). Required Test B: (0, 0) to (0.6, 0.8). Predict both distances and messages before running.
6. In your report, state which values are arguments in one call to calculateDistance(), which names are parameters, and what value replaces the function call after return.

Deliverable: C++ code, predictions, screenshots of both tests, and the parameter/argument explanation.

Task 3 [CLO 3] - Secure Lab PIN Analyzer [MODERATE | 45 minutes]

A laboratory access system classifies a numeric PIN using reusable helper functions. The aim is to practice multiple return types, Boolean functions, loops inside functions, and one function calling other functions.

Required functions:

```
int countDigits(int pin);
int digitSum(int pin);
bool isStrongPin(int pin);
```

1. countDigits() must return the number of digits in a positive integer using a loop. Do not convert the PIN to a string.
2. digitSum() must return the sum of all digits using % and integer division /.
3. isStrongPin() must return true only when all three rules are satisfied: the PIN has exactly 4 digits, its digit sum is at least 20, and its last digit is odd.
4. Inside isStrongPin(), call countDigits(pin) and digitSum(pin) instead of rewriting those calculations.
5. main() should read the PIN, display its digit count and digit sum, and then print "Strong PIN" or "Weak PIN" using the Boolean function in an if statement.
6. Required tests: 5729, 1234, and 8852. Predict the result for all three before compiling.
7. Draw a short call trace for input 5729 showing: main() -> isStrongPin() -> countDigits()/digitSum() -> return to main().

Skill check: Explain why all three prototypes are useful when the definitions are written below main().

Deliverable: C++ code, three predictions, screenshots, and the call trace.

Task 4 [CLO 3] - Smart Parcel Billing System [COMPLEX | 60 minutes]

Build one small modular program from several functions. This task deliberately combines return values, void functions, switch/if logic, call by value, and one reference parameter so you can practice the function patterns that commonly appear in midterm questions.

Required functions:

```
double calculateWeightCharge(double weightKg);
double calculateZoneCharge(int zone);
void applyMemberDiscount(double& total, char memberChoice);
void printBill(double weightCharge, double zoneCharge, double finalTotal, char
memberChoice);
```

1. Read parcel weight (kg), delivery zone (1-3), and member choice (Y/N) in main(). Reject weight <= 0 or a zone outside 1-3 before calculating the bill.
2. calculateWeightCharge() returns: Rs. 200 for weight <= 2 kg; Rs. 350 for weight > 2 and <= 5 kg; otherwise Rs. 350 + Rs. 80 for every kg above 5. Decimal extra weight should be charged proportionally.
3. calculateZoneCharge() must use a switch: Zone 1 -> Rs. 100, Zone 2 -> Rs. 200, Zone 3 -> Rs. 300.
4. In main(), calculate total = weightCharge + zoneCharge. Then call applyMemberDiscount(total, memberChoice). If the choice is Y or y, reduce total by 10%. Because total is passed by reference, the original total in main() must change.
5. printBill() receives the three amounts and member choice by value and prints a clear bill showing weight charge, zone charge, member/discount status, and final amount.
6. Required Test A: weight = 1.5, zone = 1, member = N. Required Test B: weight = 6, zone = 3, member = Y. Calculate the expected final totals before running.
7. After testing, answer these two questions in the report: (a) Why does calculateWeightCharge(weight) not change weight in main()? (b) Why can applyMemberDiscount(total, memberChoice) change total in main()?
8. Keep main() readable: input/validation -> function calls -> final display. Do not place the entire billing calculation inside main().

Deliverable: Complete modular C++ program, handwritten expected totals for both tests, screenshots, and the call-by-value vs call-by-reference explanation.

Submission Checklist

- One Word/PDF lab report containing Task 1 through Task 4 in order.
- For every task: complete code, required test output screenshot(s), and the requested prediction/trace/explanation.
- Use meaningful function names, correct prototypes, matching parameter lists, suitable return types, and proper indentation.
- Be ready to explain any function in your own code during the lab. The purpose of this lab is genuine midterm preparation, not copied code.

Task1

task1.cpp U

pf > LabTask07 > task1.cpp > printDivider()

```cpp
#include <iostream>

using namespace std;

void printDivider();

int main () {
    printDivider();
    cout << "CS110 FUNCTION PRACTICE" << endl;
    printDivider();
    cout << "Prototype -> Call -> Definition" << endl;
    printDivider();
    return 0;
}

void printDivider() {

    for(int i = 0; i < 30; i++) {
        cout << "-";
    }
    cout << endl; // adding this outside for loop so that it doesnt push hyphen to next line
}
```

Problems Output Debug Console Terminal Ports Spell Checker 1

```
tariqmahmood@Tariqs-MacBook-Air labTask07 % ./task1
------------------------------
CS110 FUNCTION PRACTICE
------------------------------
Prototype -> Call -> Definition
------------------------------
tariqmahmood@Tariqs-MacBook-Air labTask07 %
```

The function Prototype is on line #5

There are function Calls on Lines #8,#10,#12

The Function definition is on line #16

After print divider finishes executing (when called in the main program), it executes the next line of the main program (eg: after executing the function call on line #8, it should move onto executing the code on line #9)

Removing the function declaration may cause the program to not compile since the function call inside main is made before the function is declared or defined. By declaring it in the beginning, we tell the compiler to look for the function within the program even if its defined after main()

Task2

Code:

task1.cpp U task2.cpp U

pf > LabTask07 > task2.cpp > main()

```cpp
#include <iostream>
#include <cmath>

using namespace std;

double calculateDistance(double x1, double y1, double x2, double y2);
void printTravelAdvice(double distance);


int main() {
    double x1,y1,x2,y2;

    cout << "Enter a value for x1: ";
    cin >> x1;
    cout << "Enter a value for y1: ";
    cin >> y1;
    cout << "Enter a value for x2: ";
    cin >> x2;
    cout << "Enter a value for y2: ";
    cin >> y2;
    double distance = calculateDistance(x1,y1,x2,y2);
    cout << "Distance: " << distance << endl;
    printTravelAdvice(distance);
    return 0;
}

double calculateDistance(double x1, double y1, double x2, double y2) {

    return sqrt(pow(x2 - x1, 2) + pow(y2 - y1, 2));
}

void printTravelAdvice(double distance) {
    string output = (distance > 5.0) ? "Use transport": (distance > 1.0) ? "Use campus shuttle": "Walk";
    cout << output << endl;

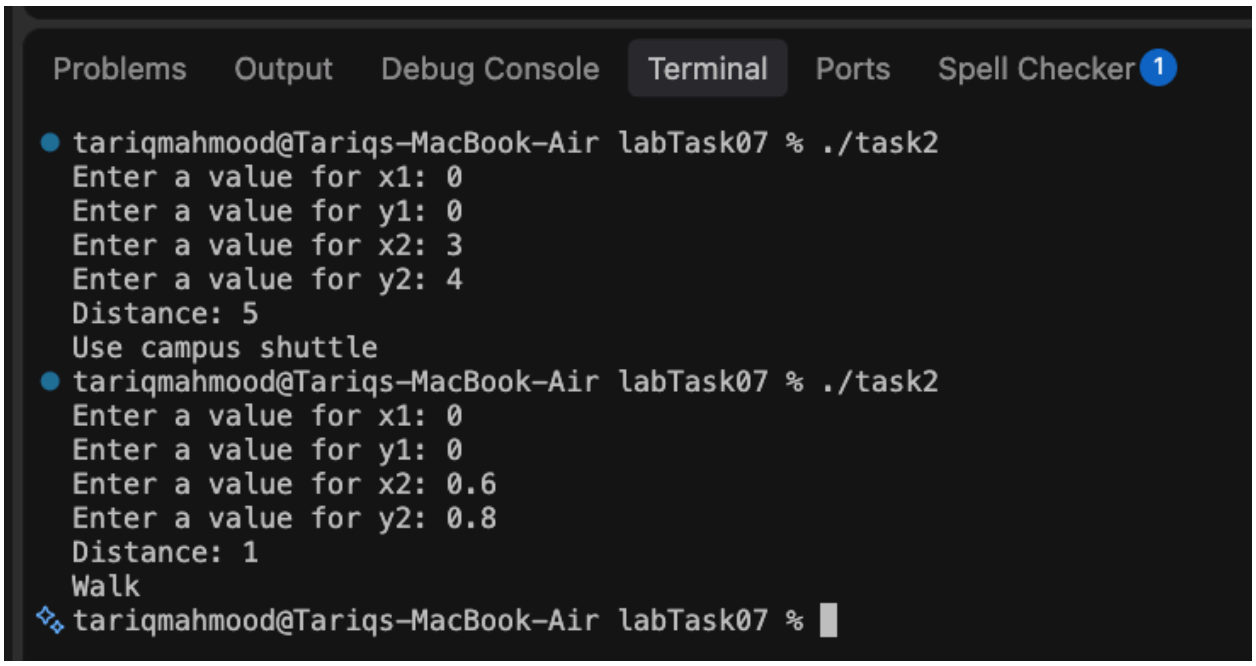
}
```

Prediction:

test1 should give distance = sqrt((4-0)^2 + (3-0)^2) = sqrt(16 + 9) = sqrt (25) = 5 so it will print use campus shuttle since distance <= 5.0

Test2 should give distance = sqrt((0.8-0)^2 + (0.6-0)^2) = sqrt (0.64 + 0.36) = sqrt (1) = 1.0 so it will print Walk since 1.0 is the max value for walk to be printed

Output:



The call in main() is calculateDistance(x1,y1,x2,y2) and the arguments are the values passed in.

For Test A those values are 0,0,3 and 4. The parameters are the names in the function definition , x1 , y1 , x2 and y2 . They are separate variables that receive values by position .

The function calculates sqrt(pow(3 - 0, 2) + pow(4 - 0, 2)) which results in 5.0 and this is used to replace the function call so the line becomes double distance = 5.0

When the function is complete, the program moves to the next line in main()

Task3

Code:

(ON NEXT PAGE)

task1.cpp U | task2.cpp U | task3.cpp U

pf > LabTask07 > task3.cpp > main()

```cpp
#include <iostream>

using namespace std;

int countDigits(int pin);
int digitSum(int pin);
bool isStrongPin(int pin);


int main() {
    int pin;

    cout << "Enter a PIN: ";
    cin >> pin;
    cout << "Digit count: " << countDigits(pin) << endl;
    cout << "Digit sum: " << digitSum(pin) << endl;
    cout << (isStrongPin(pin) ? "Strong Pin" : "Weak Pin");
    return 0;
}


int countDigits(int pin) {
    int count = 1;
    int temp = pin;
    while (temp >= 10) {
        temp /= 10;
        count += 1;
    }
    return count;
}

int digitSum(int pin) {
    int sum = 0;
    int temp = pin;
    while (temp > 0) {
        sum += temp % 10;
        temp /= 10;
    }
    return sum;
}

bool isStrongPin(int pin) {
   bool val =(countDigits(pin) == 4) ? ((digitSum(pin) >= 20) ? ((pin % 10) % 2 == 1) : false) : false;
   return val;
}
```

Predictions:

5729 should count as strong pin , (digit sum = 23 which is greater than 20 and digit count = 4 and last digit, 9, is odd

1234 weak pin, weak since last digit not odd and digit sum < 20

8852 weak pin , weak since last digit is even

OUTPUT:

```
6%
tariqmahmood@Tariqs-MacBook-Air labTask07 % clang++ task3.cpp -o task3
task3.cpp:17:59: error: expected ';' after expression
   17 |     cout << (isStrongPin(pin) ? "Strong Pin" : "Weak Pin")
      |                                                           ^
      |                                                           ;
1 error generated.
tariqmahmood@Tariqs-MacBook-Air labTask07 % clang++ task3.cpp -o task3
tariqmahmood@Tariqs-MacBook-Air labTask07 % clear
tariqmahmood@Tariqs-MacBook-Air labTask07 % ./task3
Enter a PIN: 5729
Digit count: 4
Digit sum: 23
Strong Pin%
tariqmahmood@Tariqs-MacBook-Air labTask07 % ./task3
Enter a PIN: 1234
Digit count: 4
Digit sum: 10
Weak Pin%
tariqmahmood@Tariqs-MacBook-Air labTask07 % ./task3
Enter a PIN: 8852
Digit count: 4
Digit sum: 23
Weak Pin%
tariqmahmood@Tariqs-MacBook-Air labTask07 %
```

Call Trace and Explanation of prototypes:

main() → countDigits(5729) → Returns 4 → DigitSum(5729) → returns 23 → isStrong(5729): Calls countDigits(5729) and DigitSum(5729) which return 4 and 23 respectively so isStrong(5729) → Returns True → returns to main() → prints Strong Pin.

Prototyping the functions above main is helpful since it allows those functions to be used without being defined before main().

Task4

Expected Outputs:

Test A: 1.5 kg, Zone 1, N

Weight is 1.5 so 200 is returned by calculateWeightCharge(weight) since zone is 1 and the user is not a member, calculateZoneCharge(zone) will return 100 and applyMemberDiscount(total, memberChoice) will return 0 so total is 200 + 100 = 300

Test B: 6 kg, Zone 3, Y

Weight is 6 so 350 + (6 - 5) * 80 = 350 + 80 = 430 is returned by calculateWeightCharge(weight).

Since zone is 3 calculateZoneCharge(zone) will return 300.

The user is a member, so applyMemberDiscount(total,memberChoice) will reduce the total by 10%. Therefore total is 430 + 300 = 730 and after the 10% discount the final total is 657.

Code:

k1.cpp U | task2.cpp U | task3.cpp U | task4.cpp U

LabTask07 > task4.cpp > ...

```cpp
#include <iostream>
#include <iomanip>

using namespace std;

double calculateWeightCharge(double weightKg);
double calculateZoneCharge(int zone);
void applyMemberDiscount(double& total, char memberChoice);
void printBill(double weightCharge, double zoneCharge, double finalTotal, char memberChoice);


int main() {

    double weight;
    int deliveryZone;
    char choice;
    do {
    cout << "Enter value for weight (kg): ";
    cin >> weight;
    if (weight <= 0) {
        cout << "Error, weight must be greater than 0" << endl;
    }
    }
    while (weight <= 0);

    do {
    cout << "Enter delivery zone (1-3): ";
    cin >> deliveryZone;
    if (deliveryZone < 1 || deliveryZone > 3) {
        cout << "Invalid zone, must be between 1 and 3, please try again" << endl;
    }
    }
    while (deliveryZone < 1 || deliveryZone > 3);

    do {
        cout << "Are you a member? (Y/N): ";
        cin >> choice;
        if (choice != 'Y' && choice != 'y' && choice != 'N' && choice != 'n') {
            cout << "Invalid choice, please enter Y or N" << endl;
        }
    }
    while (choice != 'Y' && choice != 'y' && choice != 'N' && choice != 'n');

    double weightCharge = calculateWeightCharge(weight);
    double zoneCharge = calculateZoneCharge(deliveryZone);
    double total = weightCharge + zoneCharge;
    applyMemberDiscount(total, choice);
    printBill(weightCharge, zoneCharge, total, choice);

    return 0;
}
```

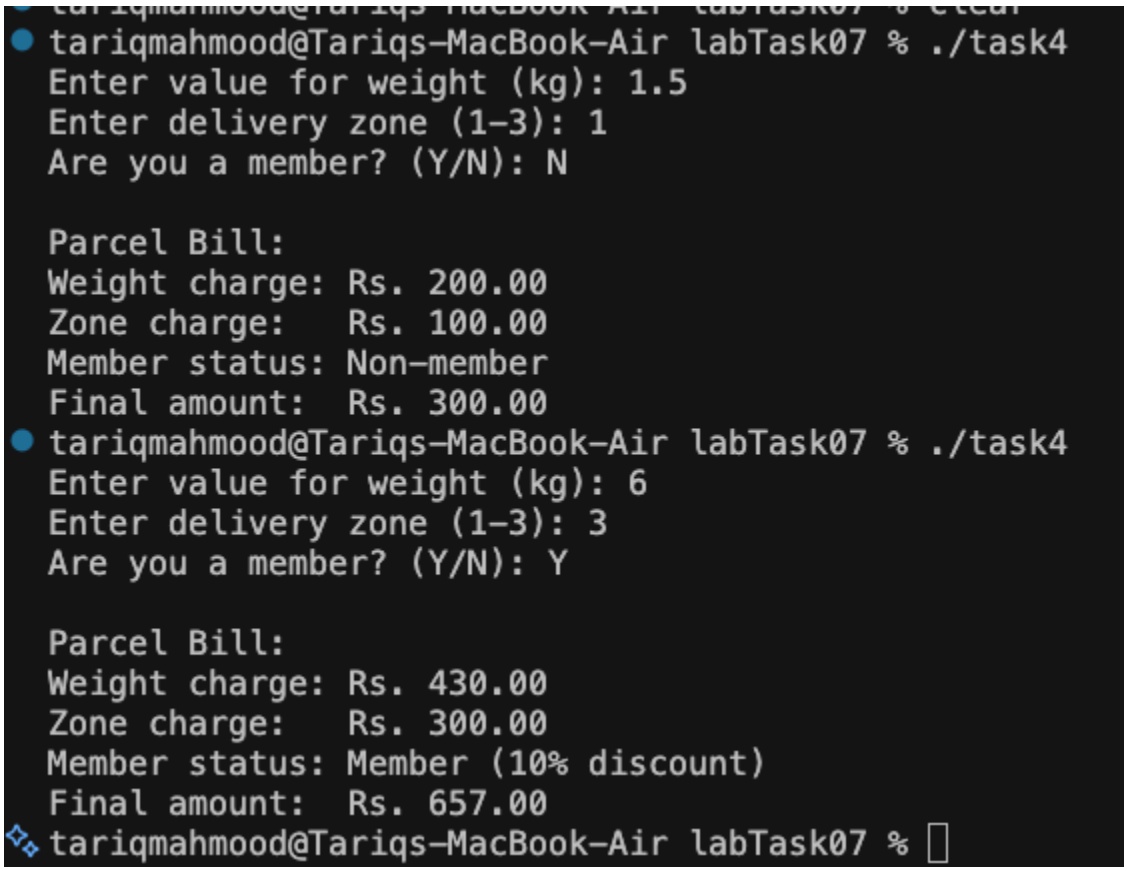
```cpp
double calculateWeightCharge(double weightKg) {
    if (weightKg <= 2) {
        return 200;
    }
    else if (weightKg <= 5) {
        return 350;
    }
    else {
        return 350 + (weightKg - 5) * 80;
    }
}

double calculateZoneCharge(int zone) {
    switch (zone) {
        case 1:
            return 100;
        case 2:
            return 200;
        case 3:
            return 300;
    }
    return 0;
}

void applyMemberDiscount(double& total, char memberChoice) {
    if (memberChoice == 'Y' || memberChoice == 'y') {
        total = total - (total * 0.10);
    }
}

void printBill(double weightCharge, double zoneCharge, double finalTotal, char memberChoice) {
    cout << fixed << setprecision(2);
    cout << endl;
    cout << "Parcel Bill: " << endl;
    cout << "Weight charge: Rs. " << weightCharge << endl;
    cout << "Zone charge:   Rs. " << zoneCharge << endl;
    cout << "Member status: " << ((memberChoice == 'Y' || memberChoice == 'y') ? "Member (10% discount)" : "Non-member") << endl;
    cout << "Final amount:  Rs. " << finalTotal << endl;
}
```

OUTPUT:



(a) calculateWeightCharge(weight) does not change weight in main() because weightKg is passed by value so the function receives a separate copy.

(b) applyMemberDiscount(total, memberChoice) can change total in main() because total is passed by reference using double&. Both names refer to the same variable.

GITHUB FOR ALL CODE / FILES:https://github.com/ARTariqDev/Nust-AI-2k26/tree/main/pf/LabTask07